

Sahoulat API Documentation

REST API for the Sahoulat mobile app (customers + professionals). Built on Laravel 12 + Sanctum. All routes are prefixed with `/api`, e.g. `https://your-domain.test/api/login`.

Tested end-to-end against the full booking → escrow payment → completion → wallet payout lifecycle, jobs/bids, emergencies, contracts, and disputes.

1. Conventions

Headers

Every request should send:

```
Accept: application/json
```

Authenticated requests must also send:

```
Authorization: Bearer <token>
```

Requests with a JSON body should send:

```
Content-Type: application/json
```

File upload endpoints (photos, KYC documents) use `multipart/form-data` instead.

Auth model

Token-based via **Laravel Sanctum**. There is no session/CSRF — every device that logs in gets its own long-lived bearer token (`device_name` you send becomes the token's label, so a user can be logged in on multiple devices and revoke them individually).

Roles

A user has exactly one `role`: `consumer`, `provider`, or `job_seeker` (job_seeker/admin flows are not covered by this API — it's scoped to the customer + professional apps). Role is fixed at registration and determines which route group (`/api/consumer/*` or `/api/provider/*`) the

account can call. Cross-role calls return `403`.

Standard error shapes

401 — missing/invalid/expired token:

```
{ "message": "Unauthenticated." }
```

403 — wrong role, or a policy denied the action:

```
{ "message": "This account type cannot access this resource." }
```

404 — model not found, or (deliberately, for security) hidden because it doesn't belong to you / isn't active.

422 — validation errors (standard Laravel shape):

```
{
  "message": "The email field is required.",
  "errors": { "email": ["The email field is required."] }
}
```

422 is also used for business-rule rejections (e.g. "That time slot is no longer available"), in the simpler `{ "message": "..."}` shape.

409 — race condition (e.g. two providers accepting the same emergency request at once).

Pagination

List endpoints that paginate return:

```
{
  "...collection key...": [ /* items */ ],
  "pagination": { "current_page": 1, "last_page": 3, "total": 42 }
}
```

Money

All amounts are numbers (PKR), not strings, e.g. `"price": 3000` not `"price": "3000.00"`.

2. Auth

POST /api/register

Auth: none **Body:**

Field	Type	Notes
name	string	required
email	string	required, unique
phone	string	required
role	string	required — consumer , provider , or job_seeker
password	string	required, min 8
password_confirmation	string	required, must match password
referral_code	string	optional
device_name	string	required — label for this token, e.g. "iPhone 15"

Response 201 :

```
{
  "user": {
    "id": 22, "name": "Test Consumer", "email": "test@example.com",
    "phone": "03001234567", "role": "consumer", "referral_code": "OG1EGG",
    "credit_balance": 0, "is_suspended": false,
    "provider_status": null,
    "created_at": "2026-07-21T12:12:07+00:00"
  },
  "token": "1|dFx1zYRMLAFDEVy2Ba1JMLVvxzDkRPhobybCfHbN4541fa29"
}
```

provider_status is only meaningful when role is provider (mirrors provider_profiles.status : draft → pending → approved / rejected).

POST /api/login

Auth: none **Body:** email , password , device_name (all required) **Response 200 :** same shape as register. **Errors:** 422 if credentials don't match, or if the account is suspended.

POST /api/logout

Auth: required — revokes only the token used for this request (this device). **Response:** { "message": "Logged out." }

POST /api/logout-all

Auth: required — revokes every token for the user (all devices). **Response:** { "message": "Logged out of all devices." }

GET /api/me

Auth: required **Response:** { "user": { ...same shape as register... } }

PUT /api/profile

Auth: required **Body:** `name` (required), `email` (required, unique except self), `phone` (optional) **Response:** { "user": {...} }

PUT /api/profile/password

Auth: required **Body:** `current_password` (required, must match), `password` (required, confirmed, min 8), `password_confirmation` **Response:** { "message": "Password updated." }

3. Shared / public browsing

None of these require auth unless noted.

GET /api/categories

Active categories with their active services (the "browse services" catalog, cached 1h server-side). **Response:**

```
{
  "categories": [
    {
      "id": 1, "name": "AC Repair & Service", "slug": "ac-repair-service",
      "description": "...", "icon": "ac", "image_url": null, "banner_url": null,
      "services": [
        { "id": 2, "category_id": 1, "name": "AC Gas Refill", "slug": "ac-gas-refill",
          "description": "...", "thumbnail_url": null, "base_price": 3500,
          "duration_minutes": 90, "is_active": true }
      ]
    }
  ]
}
```

```

    ]
  }
]
}

```

GET /api/services

All active services (flat list, e.g. for search/typeahead). Response: `{ "services": [ {...ServiceResource...} ] }`

GET /api/services/{slug}

Service detail + approved providers offering it (cheapest first, paginated) + 3 related services. **Response:**

```

{
  "service": { "id": 2, "name": "AC Gas Refill", "...": "..." },
  "providers": [
    { "provider_profile_id": 21, "provider": { "...ProviderProfileResource..." },
    "price": 3000 }
  ],
  "providers_pagination": { "current_page": 1, "last_page": 1, "total": 1 },
  "related_services": [ {...} ]
}

```

GET /api/providers

Public provider directory. **Query:** `q` (search name/bio/city/service), `city` (exact match). Both optional. **Response:**

```

{
  "providers": [ {...ProviderProfileResource...} ],
  "pagination": { "current_page": 1, "last_page": 1, "total": 5 },
  "cities": ["Islamabad", "Karachi", "Lahore"]
}

```

GET /api/providers/{id}

One provider's public profile: bio, services offered (with prices), portfolio photos, and latest 10 reviews. **Response:** `{ "provider": {...}, "reviews": [ { "id":1, "rating":5, "comment":"...", "consumer_name":"...", "service_name":"...", "created_at":"..." } ] }`

GET /api/providers/{id}/services/{serviceId}/availability

Bookable dates + time slots for a specific provider/service pairing — call this before showing the booking picker. **Query:** `date` (optional, `YYYY-MM-DD` ; defaults to the first bookable date).

Response:

```
{
  "price": 3000, "duration_minutes": 90,
  "dates": [
    { "value": "2026-07-21", "label": "Today – 21 Jul" },
    { "value": "2026-07-22", "label": "Wed, 22 Jul" }
  ],
  "selected_date": "2026-07-21",
  "slots": [
    { "value": "09:00", "label": "9:00 AM", "available": false },
    { "value": "10:00", "label": "10:00 AM", "available": true }
  ]
}
```

GET /api/subscription-plans

Active maintenance/AMC plans. Response: `{ "plans": [ { "id":1, "name":"...", "slug":"...", "service": {...}, "frequency_months":1, "frequency_label":"Monthly", "total_visits":12, "price_per_visit":2000, "is_active":true } ] }`

GET /api/cities

Cities currently served (for city pickers / client-side coverage checks). Response: `{ "cities": ["Islamabad", "Karachi", "Lahore"] }`

4. Shared / authenticated (any role)

All require `Authorization: Bearer <token>`.

GET /api/notifications

Paginated, newest first. Response: `{ "notifications": [ {...} ], "unread_count": 3, "pagination": {...} }` Each notification: `{ "id", "type", "title", "body", "url", "is_read", "read_at", "created_at" }`

POST /api/notifications/{id}/read

Marks one notification read. Response: `{ "notification": {...} }`

POST `/api/notifications/read-all`

Response: `{ "message": "All notifications marked as read." }`

GET `/api/bookings/{id}/room`

Chat history + latest tracking pin for a booking (both consumer and provider on the booking can call this). **Response:**

```
{
  "is_communicable": true,
  "can_share_location": true,
  "messages": [ { "id":1, "sender_id":22, "sender_name":"Test Consumer", "body":"...",
"created_at":"..." } ],
  "latest_tracking": { "id":1, "latitude":24.86, "longitude":67.01, "note":null,
"created_at":"..." }
}
```

POST `/api/bookings/{id}/messages`

Body: `body` (string, required, max 2000) **Response** **201**: `{ "message_data": {...} }` — broadcast in realtime over the existing Reverb/Echo channel for the booking.

POST `/api/bookings/{id}/tracking`

Post a live location pin (typically the provider en route). **Body:** `latitude`, `longitude` (required), `note` (optional). **Response** **201**: `{ "tracking": {...} }`

POST `/api/bookings/{id}/dispute`

Body: `reason` (string, required, max 2000) **Response** **201**: `{ "dispute": {...} }`

GET `/api/disputes/{id}`

Response: `{ "dispute": {
 "id","reference","opened_by_role","reason","status","resolution","resolution_note","resolved_at","created_at","updated_at"
} }`

5. Consumer API (`/api/consumer/*`, role `consumer` required)

GET /api/consumer/dashboard

Home-screen summary. Response: `{ "recent_bookings": [ {...BookingResource, up to 5...} ] }`

Addresses

Method	Path	Body	Notes
GET	/addresses	-	<code>{ "addresses": [...]</code> <code>}</code>
POST	/addresses	label , address , city (required); latitude , longitude , is_default (optional)	201 , <code>{ "address":</code> <code>{...} }</code>
PUT	/addresses/{id}	same as POST	<code>{ "address": {...} }</code>
DELETE	/addresses/{id}	-	<code>{ "message":</code> <code>"Address removed." }</code>

Address shape: `{ "id","label","address","city","latitude","longitude","is_default" }`

Bookings (direct booking flow)

GET /bookings — paginated, newest first. `{ "bookings": [...], "pagination": {...} }`

POST /bookings — create a direct booking with a specific approved provider.

Field	Required	Notes
provider_profile_id	yes	must be an approved provider
service_id	yes	must be a service that provider actively offers
scheduled_date	yes	one of the values from the availability endpoint
scheduled_time	yes	HH:MM , must be an available slot
address	yes	
latitude , longitude	no	
notes	no	max 1000

Response 201 : `{ "booking": {...BookingResource...} }`. 422 if the provider is outside served cities or the slot just got taken.

GET /bookings/{id} — { "booking": {...} } (full detail incl. `payments`, `review`, `dispute`)

POST /bookings/{id}/cancel — only while cancellable by the consumer (not yet in progress).

Auto-refunds escrow if already paid. { "message": "...", "booking": {...} }

GET /bookings/{id}/payment-options — { "gateways": [{"key":"mock","label":"Test payment (sandbox)"}], "max_credit_applicable": 0, "amount": 3000 }

POST /bookings/{id}/pay — Body: `gateway` (required unless referral credit fully covers the amount — currently only `"mock"` is enabled by default; `jazzcash` / `easypaisa` activate via env config), `apply_credit` (bool, optional — applies available referral credit first). Response `201`: { "message": "...", "payment": {...} }, or, for gateways needing an off-site redirect: { "status": "pending", "redirect_url": "...", "redirect_fields": {...} }.

POST /bookings/{id}/release — release escrow to the provider once completed & undisputed. { "message": "...", "booking": {...} }

POST /bookings/{id}/review — Body: `rating` (1-5, required), `comment` (optional, max 1000). Only once, only after completion. Response `201`: { "review": {...} }

Booking resource shape (used throughout)

```
{
  "id": 1, "reference": "BK-Z6POMA", "status": "confirmed",
  "service": { "...ServiceResource..." },
  "provider": { "...ProviderProfileResource..." },
  "consumer": { "id": 22, "name": "Test Consumer", "phone": "03001234567" },
  "scheduled_date": "2026-07-22", "scheduled_time": "10:00",
  "price": 3000, "duration_minutes": 90, "address": "House 1, Karachi",
  "latitude": null, "longitude": null, "notes": null,
  "cancelled_by": null, "cancellation_reason": null,
  "confirmed_at": "...", "started_at": null, "completed_at": null, "cancelled_at":
  null,
  "created_at": "...",
  "payments": [ {...} ], "review": null, "dispute": null,
  "permissions": {
    "can_cancel": true, "is_payable": true, "is_reviewable": false,
    "is_disputable": false, "is_communicable": true, "can_share_location": false,
    "is_provider": false
  }
}
```

`permissions` tells the app which action buttons to show — always trust this over re-deriving the logic client-side.

Jobs (post & bid flow)

GET /jobs — `{ "jobs": [...], "pagination": {...} }`

POST /jobs — post a job for open bidding. Multipart if attaching photos.

Field	Required
<code>service_id</code>	yes
<code>description</code>	yes, max 2000
<code>budget</code>	no, numeric
<code>preferred_date</code>	no, date $\geq$ today
<code>address</code> , <code>city</code>	yes
<code>latitude</code> , <code>longitude</code>	no
<code>photos[]</code>	no, up to 5 images, 5MB each

Response `201`: `{ "job": {...JobPostResource...} }`

GET /jobs/{id} — full detail with all bids (each bid includes the bidding provider's profile).

POST /jobs/{id}/cancel — only while `status = open`. Rejects any pending bids too.

POST /jobs/{id}/bids/{bidId}/accept — accepts a bid, creates a confirmed `Booking`, rejects all other pending bids on that job, notifies the winning + losing providers. `{ "message": "...", "booking": {...} }`. `422` if the slot clashes with another confirmed booking for that provider, or the proposed time has passed.

JobPost resource shape

```
{
  "id": 1, "reference": "JOB-PZYQ40", "status": "open",
  "service": {...}, "consumer": { "id":22, "name":"..." },
  "description": "...", "budget": null, "preferred_date": null,
  "address": "...", "latitude": null, "longitude": null, "city": "Karachi",
  "photos": [ { "id":1, "url":"https://.../job-photos/1/xyz.jpg" } ],
  "bids_count": 2, "pending_bids_count": 1,
```

```

"bids": [ {...BidResource...} ],
"my_bid": null,
"awarded_at": null, "created_at": "..."
}

```

Contracts (multi-service projects, admin-quoted)

GET /contracts — { "contracts": [...], "pagination": {...} }

POST /contracts — submit for a manual admin quote. Multipart if attaching photos.

Field	Required
title, description	yes
preferred_start_date	no
address, city	yes
latitude, longitude	no
photos[]	no, up to 8 images
items[]	yes, min 1 — each: service_id (required), quantity (required, 1-100), notes (optional)

Response 201: { "contract": {...} } — status starts as submitted ; an admin later sets quoted_total and per-milestone amounts.

GET /contracts/{id} — full detail incl. items, photos, milestones.

POST /contracts/{id}/accept / **POST /contracts/{id}/reject** — only while status = quoted.

POST /contracts/{id}/cancel — only while cancellable.

GET /contracts/{id}/milestones/{milestoneId}/payment-options — same shape as booking payment-options.

POST /contracts/{id}/milestones/{milestoneId}/pay — same body/response shape as booking /pay.

Emergencies (SOS)

GET /emergencies — { "emergencies": [...], "pagination": {...} }

POST /emergencies — broadcasts to nearby approved providers offering that service in that city (realtime + notification).

Field	Required
<code>service_id</code>	yes, must be active
<code>address</code> , <code>city</code>	yes
<code>notes</code>	no

Response 201: { "emergency": {...} }

GET /emergencies/{id} — includes `matched_provider` and `booking` once accepted.

POST /emergencies/{id}/cancel — only while still `open`.

Subscriptions (maintenance plans)

GET /subscriptions — { "subscriptions": [...], "pagination": {...} }

POST /subscription-plans/{planSlug}/subscribe Body: `address`, `city` (required), `latitude`, `longitude` (optional), `start_date` (required, date ≥ today). Response 201: { "subscription": {...} } — status starts `pending_assignment` until admin assigns a provider.

GET /subscriptions/{id} — includes `provider` and past `bookings` once active.

POST /subscriptions/{id}/cancel — Body: `reason` (optional). Already-scheduled visits are unaffected.

6. Provider API (/api/provider/*, role provider required)

Most endpoints beyond onboarding require an **approved** `ProviderProfile` — calling them before approval returns empty collections (list endpoints) or a 403 from the `actAsApprovedProvider` gate (action endpoints).

GET /api/provider/dashboard

Home-screen summary: counters, wallet balances, 6-month earnings trend, completion rate, average response time, bid win rate, today's schedule.

```
{
  "profile": {...ProviderProfileResource... or null},
  "pending_bookings": 0, "available_jobs": 2, "open_emergencies": 1,
  "wallet_available": 2700, "wallet_escrow": 0,
```

```

"earnings_month": 2700, "earnings_total": 2700, "earnings_delta": 100,
"earnings_series": [ {"label":"Feb","value":0}, "...", {"label":"Jul","value":2700}
],
"jobs_completed": 1, "active_bookings": 1, "completion_rate": 100,
"response_minutes": 0, "bid_win_rate": 100, "bids_pending": 0,
"today_schedule": [ {...BookingResource...} ]
}

```

Onboarding / KYC

GET /onboarding — profile + uploaded documents + step progress + what's missing.

```

{
  "profile": {...},
  "documents": [ { "id":1, "type":"cnic_front", "original_name":"...",
"download_url":"...", "created_at":"..." } ],
  "document_types": { "cnic_front": {"label":"CNIC – Front","required":true}, "...":
"..."},
  "steps": [
    { "label": "Your details", "done": true },
    { "label": "KYC documents", "done": false, "uploaded": 1, "required": 3 },
    { "label": "Review", "done": false, "submitted": false }
  ],
  "missing": ["CNIC – Back", "Selfie holding CNIC"],
  "can_submit": false
}

```

PUT /onboarding — Body: `experience_years`, `city`, `cnic_number` (required); `business_name`, `bio`, `address`, `latitude`, `longitude` (optional). Blocked (422) once submitted/approved.

POST /onboarding/documents — multipart. Body: `type` (one of the `document_types` keys), `file` (jpg/jpeg/png/pdf, max 4MB). Replaces any existing document of the same type. Response 201: { "document": {...} }

GET /onboarding/documents/{id} — streams the private file (owner only). Not JSON — returns the raw file.

DELETE /onboarding/documents/{id} — { "message": "Document removed." }

POST /onboarding/submit — moves profile to `pending` for admin review once details + all required documents are present. 422 with a listing of what's missing otherwise.

Services offered

Method	Path	Body	Notes
GET	/services	-	{ "offered": [...], "available": [...], "booking_counts": {...} } — empty until approved
POST	/services	service_id (required, not already offered), price (required)	201, { "provider_service": {...} }
PUT	/services/{id}	price (required), is_active (optional bool)	{ "provider_service": {...} }
DELETE	/services/{id}	-	{ "message": "..."} }

Bookings

GET /bookings — **Query:** status (all | pending | confirmed | in_progress | completed | cancelled , default all), q (search reference/address/consumer/service name). Response: { "bookings": [...], "counts": {"all":5,"pending":1,...}, "filter": "all", "pagination": {...} }

GET /bookings/{id} — full detail.

POST /bookings/{id}/status — Body: action (required — confirm | decline | start | complete | cancel), cancellation_reason (optional, used for decline / cancel). Valid transitions: pending →(confirm/decline)→ , confirmed →(start/cancel)→ , in_progress →(complete)→ .

Declining/cancelling auto-refunds escrow. Response: { "message": "...", "booking": {...} }

Jobs & bids

GET /jobs — open jobs matching services this provider offers; each job includes my_bid (null if not yet bid).

GET /jobs/{id} — { "job": {...}, "offers_service": true, "slot_options": [{"value":"09:00","label":"9:00 AM"}, ...] }

POST /jobs/{id}/bids — Body: amount (required, numeric), proposed_date (required, ≥ today), proposed_time (required, one of slot_options), message (optional). One bid per provider per job. Response 201 : { "bid": {...} }

GET /bids — **Query:** `status` (`all` | `pending` | `accepted` | `rejected` | `withdrawn`). Response: `{ "bids": [...], "counts": {...}, "win_rate": 100, "pipeline": 2800 }` (`pipeline` = sum of pending bid amounts).

PUT /bids/{id} — same body as create; only while pending and the job is still open.

DELETE /bids/{id} — withdraws a pending bid.

Emergencies

GET /emergencies — open requests in the provider's city for services they offer; each includes `my_price` (this provider's price for that service).

POST /emergencies/{id}/accept — first to accept wins; creates a confirmed booking immediately. `409` if another provider already claimed it.

Wallet & payouts

GET /wallet — **Query:** `bucket` (`all` | `available` | `escrow`). Balances, lifetime/monthly earnings, 6-month trend, ledger entries, and recent withdrawal requests.

```
{
  "wallet": { "id":1, "available_balance": 2700, "escrow_balance": 0 },
  "total_earned": 2700, "earned_this_month": 2700,
  "earnings_series": [ { "label":"Feb","value":0}, "... " ],
  "ledger": {
    "bucket": "all", "counts": {"all":3,"available":1,"escrow":2},
    "entries": [ { "id":3, "bucket":"available", "type":"...", "amount":2700,
  "description":"...", "booking_reference":"BK-Z6POMA", "created_at":"..." } ],
    "pagination": {...}
  },
  "min_withdrawal": 500,
  "withdrawal_requests": [ { "id":1, "reference":"WD-...", "amount":2700,
  "status":"pending", "payout_method":"jazzcash", "method_label":"JazzCash",
  "admin_notes":null, "processed_at":null, "created_at":"..." } ]
}
```

POST /payout-method — Body: `payout_method` (`bank` | `jazzcash` | `easypaisa` , required), `payout_account_title` , `payout_account_number` (required), `payout_bank_name` (required only if `payout_method = bank`). Response: `{ "profile": {...} }`

POST /withdrawals — Body: `amount` (required, $\geq$ `min_withdrawal`, $\leq$ current available balance). Requires a payout method already saved. Response 201: `{ "message": "...", "withdrawal": {...} }`

Portfolio

Method	Path	Body	Notes
GET	<code>/portfolio</code>	-	<code>{ "photos": [...] }</code>
POST	<code>/portfolio</code>	multipart: <code>photos[]</code> (images, up to remaining slots under a 12-photo cap), <code>caption</code> (optional)	201, <code>{ "photos": [...just the new ones...] }</code>
DELETE	<code>/portfolio/{photoId}</code>	-	<code>{ "message": "Photo removed." }</code>

7. Resource reference

Quick field reference for nested objects that recur throughout the API.

ProviderProfileResource

```
{
  "id": 21, "user_id": 23, "name": "Test Pro", "business_name": null,
  "display_name": "Test Pro", "bio": null, "experience_years": 5,
  "rating_avg": 5.0, "reviews_count": 1, "city": "Karachi", "address": null,
  "latitude": null, "longitude": null, "status": "approved", "rejection_reason": null,
  "has_payout_method": true,
  "services": [ {...ProviderServiceResource, when loaded...} ],
  "portfolio": [ {...ProviderPortfolioPhotoResource, when loaded...} ]
}
```

`status` $\in$ `draft` | `pending` | `approved` | `rejected`.

ServiceResource — `{ "id", "category_id", "category": {...or omitted}, "name", "slug", "description", "thumbnail_url", "base_price", "duration_minutes", "is_active" }`

BidResource — `{ "id", "job_post_id", "job_post": {...summary...}, "provider": {...ProviderProfileResource...}, "amount", "p" }`


```
} . status ∈ pending | accepted | rejected | withdrawn .
```

ContractResource — {

```
"id", "reference", "title", "description", "address", "latitude", "longitude", "city", "preferred_start_date"  
} . status ∈ submitted | quoted | accepted | rejected | in_progress | completed |  
cancelled .
```

EmergencyRequestResource — {

```
"id", "reference", "status", "service", "consumer", "address", "city", "notes", "booking_id", "booking", "match"  
} . status ∈ open | matched | cancelled . my_price only populated on the provider board  
endpoint.
```

SubscriptionResource — {

```
"id", "reference", "status", "plan", "provider", "address", "city", "next_visit_date", "visits_used", "is_canc"  
} . status ∈ pending_assignment | active | cancelled | completed .
```

PaymentResource — {

```
"id", "reference", "gateway", "amount", "credit_applied", "status", "paid_at", "released_at", "refunded_at"  
} . status ∈ pending | escrow | released | refunded | failed .
```

DisputeResource — {

```
"id", "reference", "opened_by_role", "reason", "status", "resolution", "resolution_note", "resolved_at", "cre"  
} . status ∈ open | resolved | dismissed (resolution is set by an admin).
```

8. What's intentionally out of scope here

- **Admin panel** — stays web-only; not part of this API.
- **Job seeker (careers/recruitment) flows** — the mobile app is for customers + professionals, not the internal hiring board.
- **Corporate/B2B accounts, referrals dashboard** — not wired into this API pass; can be added the same way (they already have web controllers to mirror) if the app needs them later.
- **Push notification delivery** — this API stores/reads in-app `Notification` rows (badge counts, notification center), but does not yet register Expo/FCM/APNs device tokens or send pushes. That's a separate small piece of work (a `device_tokens` table + a dispatch step in `Notifier`) worth doing once the app shell exists.

- **Real payment gateways** — JazzCash/Easypaisa drivers exist server-side but are disabled by default (`config/payments.php`); only the `mock` gateway is enabled out of the box, same as the web app.

9. Testing this yourself

```
curl -X POST https://your-domain/api/register \
  -H "Accept: application/json" -H "Content-Type: application/json" \
  -d
'{"name":"Jane","email":"jane@example.com","phone":"03001234567","role":"consumer","password":"password123","password_confirmation":"password123","device_name":"iPhone"}'
```

then, using the returned token:

```
curl https://your-domain/api/me \
  -H "Accept: application/json" -H "Authorization: Bearer <token>"
```

Every endpoint in this document was hit with real requests against a local instance during development (full booking → escrow → completion → wallet release cycle, jobs/bids, emergency accept, contract creation, dispute open, and token revocation on logout all verified working).